

条款 19: 设计 class 犹如设计 type

Treat class design as type design.

C++ 就像在其他 OOP（面向对象编程）语言一样，当你定义一个新 class，也就定义了一个新 type。身为 C++ 程序员，你的许多时间主要用来扩张你的类型系统（type system）。这意味你并不只是 class 设计者，还是 type 设计者。重载（overloading）函数和操作符、控制内存的分配和归还、定义对象的初始化和终结……全都在你手上。因此你应该带着和“语言设计者当初设计语言内置类型时”一样的谨慎来研讨 class 的设计。

设计优秀的 classes 是一项艰巨的工作，因为设计好的 types 是一项艰巨的工作。好的 types 有自然的语法，直观的语义，以及一或多个高效实现品。在 C++ 中，一个不良规划下的 class 定义恐怕无法达到上述任何一个目标。甚至 class 的成员函数的效率都有可能受到它们“如何被声明”的影响。

那么，如何设计高效的 classes 呢？首先你必须了解你面对的问题。几乎每一个 class 都要求你面对以下提问，而你的回答往往导致你的设计规范：

- **新 type 的对象应该如何被创建和销毁？** 这会影响到你的 class 的构造函数和析构函数以及内存分配函数和释放函数（`operator new`, `operator new[]`, `operator delete` 和 `operator delete[]`——见第 8 章）的设计，当然前提是如果你打算撰写它们。
- **对象的初始化和对象的赋值该有什么样的差别？** 这个答案决定你的构造函数和赋值（*assignment*）操作符的行为，以及其间的差异。很重要的是别混淆了“初始化”和“赋值”，因为它们对应于不同的函数调用（见条款 4）。
- **新 type 的对象如果被 *passed by value*（以值传递），意味着什么？** 记住，*copy* 构造函数用来定义一个 type 的 *pass-by-value* 该如何实现。
- **什么是新 type 的“合法值”？** 对 class 的成员变量而言，通常只有某些数值集是有效的。那些数值集决定了你的 class 必须维护的约束条件（*invariants*），也就决定了你的成员函数（特别是构造函数、赋值操作符和所谓“setter”函数）必须进行的错误检查工作。它也影响函数抛出的异常、以及（极少被使用的）函数异常明细列（*exception specifications*）。

- 你的新 **type** 需要配合某个继承图系 (**inheritance graph**) 吗? 如果你继承自某些既有的 **classes**, 你就受到那些 **classes** 的设计的束缚, 特别是受到“它们的函数是 **virtual** 或 **non-virtual**”的影响 (见条款 34 和条款 36)。如果你允许其他 **classes** 继承你的 **class**, 那会影响你所声明的函数——尤其是析构函数——是否为 **virtual** (见条款 7)。
- 你的新 **type** 需要什么样的转换? 你的 **type** 生存于其他一海票 **types** 之间, 因而彼此该有转换行为吗? 如果你希望允许类型 **T1** 之物被隐式转换为类型 **T2** 之物, 就必须在 **class T1** 内写一个类型转换函数 (**operator T2**) 或在 **class T2** 内写一个 **non-explicit-one-argument** (可被单一实参调用) 的构造函数。如果你只允许 **explicit** 构造函数存在, 就得写出专门负责执行转换的函数, 且不得为类型转换操作符 (**type conversion operators**) 或 **non-explicit-one-argument** 构造函数。(条款 15 有隐式和显式转换函数的范例。)
- 什么样的操作符和函数对此新 **type** 而言是合理的? 这个问题的答案决定你将为你的 **class** 声明哪些函数。其中某些该是 **member** 函数, 某些则否 (见条款 23, 24, 46)。
- 什么样的标准函数应该驳回? 那些正是你必须声明为 **private** 者 (见条款 6)。
- 谁该取用新 **type** 的成员? 这个提问可以帮助你决定哪个成员为 **public**, 哪个为 **protected**, 哪个为 **private**。它也帮助你决定哪一个 **classes** 和/或 **functions** 应该是 **friends**, 以及将它们嵌套于另一个之内是否合理。
- 什么是新 **type** 的“未声明接口” (**undeclared interface**)? 它对效率、异常安全性 (见条款 29) 以及资源运用 (例如多任务锁定和动态内存) 提供何种保证? 你在这些方面提供的保证将为你的 **class** 实现代码加上相应的约束条件。
- 你的新 **type** 有多么一般化? 或许你其实并非定义一个新 **type**, 而是定义一整个 **types** 家族。果真如此你就不该定义一个新 **class**, 而是应该定义一个新的 **class template**。

- 你真的需要一个新 **type** 吗？如果只是定义新的 **derived class** 以便为既有的 **class** 添加机能，那么说不定单纯定义一或多个 **non-member 函数** 或 **templates**，更能够达到目标。

这些问题不容易回答，所以定义出高效的 **classes** 是一种挑战。然而如果能够设计出至少像 C++ 内置类型一样好的用户自定义（**user-defined**）**classes**，一切汗水便都值得。

请记住

- **Class** 的设计就是 **type** 的设计。在定义一个新 **type** 之前，请确定你已经考虑过本条款覆盖的所有讨论主题。

条款 20: 宁以 **pass-by-reference-to-const** 替换 **pass-by-value**

Prefer **pass-by-reference-to-const** to **pass-by-value**.

缺省情况下 C++ 以 **by value** 方式（一个继承自 C 的方式）传递对象至（或来自）函数。除非你另外指定，否则函数参数都是以实际实参的复件（副本）为初值，而调用端所获得的亦是函数返回值的一个复件。这些复件（副本）系由对象的 **copy** 构造函数产出，这可能使得 **pass-by-value** 成为昂贵的（费时的）操作。考虑以下 **class** 继承体系：

```
class Person {
public:
    Person();                      //为求简化，省略参数
    virtual ~Person();             //条款 7 告诉你为什么它是 virtual
    ...
private:
    std::string name;
    std::string address;
};

class Student: public Person {
public:
    Student();                     //再次省略参数
    ~Student();
    ...
private:
    std::string schoolName;
    std::string schoolAddress;
};
```